

SHA-1

Материал из Википедии — свободной энциклопедии

Secure Hash Algorithm 1 — алгоритм криптографического хеширования. Описан в RFC 3174. Для входного сообщения произвольной длины (максимум **2⁶⁴ — 1** бит, что примерно равно 2 эксабайта) алгоритм генерирует 160-битное (20 байт) хеш-значение, называемое также дайджестом сообщения, которое обычно отображается как шестнадцатеричное число длиной в 40 цифр. Используется во многих криптографических приложениях и протоколах. Также рекомендован в качестве основного для государственных учреждений в США. Принципы, положенные в основу SHA-1, аналогичны тем, которые использовались Рональдом Ривестом при проектировании MD4.

	SHA-1
Разработчики	NSA совместно с NIST
Создан	1995
Опубликован	1995
Предшественник	SHA-0
Преемник	SHA-2
Размер хеша	160 бит
Число раундов	80
Тип	хеш-функция

Содержание

История

Описание алгоритма

Инициализация

Главный цикл

Псевдокод SHA-1

Примеры

Криптоанализ

SHAppending

Реальный взлом: SHAttered (нахождение коллизий)

Сравнение SHA-1 с другими алгоритмами

Сравнение с MD5

Сравнение с ГОСТ Р 34.11-94

Сравнение с другими SHA

Использование

Отказ от использования

Примечания

Литература

Ссылки

История

В 1993 году NSA совместно с NIST разработали алгоритм безопасного хеширования (сейчас известный как SHA-0) (опубликован в документе *FIPS PUB 180*) для стандарта безопасного хеширования. Однако вскоре NSA отозвало данную версию, сославшись на обнаруженную ими ошибку, которая так и не была раскрыта. И заменило его исправленной версией, опубликованной в 1995 году в документе *FIPS PUB 180-1*. Эта версия и считается тем, что называют SHA-1. Позже, на конференции CRYPTO в 1998 году два французских исследователя представили атаку на алгоритм SHA-0, которая не работала на алгоритме SHA-1. Возможно, это и была ошибка, открытая NSA.

Описание алгоритма

SHA-1 реализует хеш-функцию, построенную на идее функции сжатия. Входами функции сжатия являются блок сообщения длиной 512 бит и выход предыдущего блока сообщения. Выход представляет собой значение всех хеш-блоков до этого момента. Иными словами, хеш-блок M_i равен $h_i = f(M_i, h_{i-1})$. Хеш-значением всего сообщения является выход последнего блока.

Инициализация

Исходное сообщение разбивается на блоки по 512 бит в каждом. Последний блок дополняется до длины, кратной 512 бит. Сначала добавляется 1 (бит), а потом — нули, чтобы длина блока стала равной $512 - 64 = 448$ бит. В оставшиеся 64 бита записывается длина исходного сообщения в битах (в big-endian формате). Если последний блок имеет длину более 447, но менее 512 бит, то дополнение выполняется следующим образом: сначала добавляется 1 (бит), затем — нули вплоть до конца 512-битного блока; после этого создается ещё один 512-битный блок, который заполняется вплоть до 448 бит нулями, после чего в оставшиеся 64 бита записывается длина исходного сообщения в битах (в big-endian формате). Дополнение последнего блока осуществляется всегда, даже если сообщение уже имеет нужную длину.

Инициализируются пять 32-битовых переменных.

A = 0x67452301

B = 0xEFCDAB89

C = 0x98BADCFE

D = 0x10325476

E = 0xC3D2E1F0

Определяются четыре нелинейные операции и четыре константы.

$F_t(m, l, k) = (m \wedge l) \vee (\neg m \wedge k)$	$K_t = 0x5A827999$	$0 \leq t \leq 19$
$F_t(m, l, k) = m \oplus l \oplus k$	$K_t = 0x6ED9EBA1$	$20 \leq t \leq 39$
$F_t(m, l, k) = (m \wedge l) \vee (m \wedge k) \vee (l \wedge k)$	$K_t = 0x8F1BBCDC$	$40 \leq t \leq 59$
$F_t(m, l, k) = m \oplus l \oplus k$	$K_t = 0xCA62C1D6$	$60 \leq t \leq 79$

Главный цикл

Главный цикл итеративно обрабатывает каждый 512-битный блок. В начале каждого цикла вводятся переменные a, b, c, d, e, которые инициализируются значениями A, B, C, D, E, соответственно. Блок сообщения преобразуется из 16 32-битовых слов M_i в 80 32-битовых слов W_j по следующему правилу:

$W_t = M_t$

при $0 \leq t \leq 15$

$W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \ll 1$

при $16 \leq t \leq 79$

, где « $\ll$ » — это циклический сдвиг влево.

для t от 0 до 79

temp = (a $\ll$ 5) + $F_t(b, c, d)$ + e + W_t + K_t

e = d

d = c

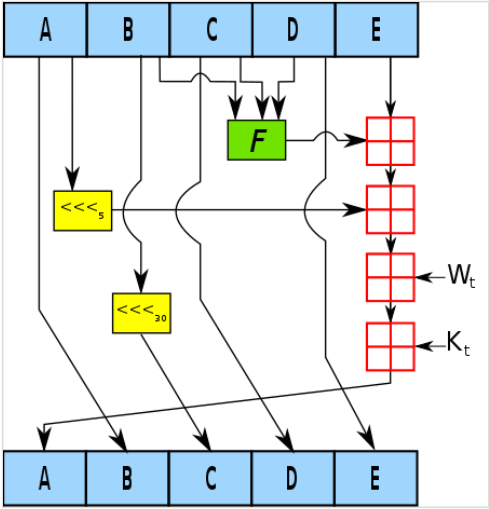
c = b $\ll$ 30

b = a

a = temp

, где «+» — сложение беззнаковых 32-битных целых чисел с отбрасыванием избытка (33-го бита).

После этого к A, B, C, D, E прибавляются значения a, b, c, d, e, соответственно. Начинается следующая итерация.



Итоговым значением будет объединение пяти 32-битовых слов (A, B, C, D, E) в одно 160-битное хеш-значение.

Псевдокод SHA-1

Псевдокод алгоритма SHA-1 следующий:

Замечание: Все используемые переменные 32 бита.

Инициализация переменных:
h0 = 0x67452301
h1 = 0xEFCDAB89
h2 = 0x98BADCFE
h3 = 0x10325476
h4 = 0xC3D2E1F0

Предварительная обработка:
Присоединяем бит '1' к сообщению
Присоединяем k битов '0', где k наименьшее число ≥ 0 такое, что длина получившегося сообщения (в битах) сравнима по модулю 512 с 448 (length mod 512 == 448)
Добавляем длину исходного сообщения (до предварительной обработки) как целое 64-битное Big-endian число, в битах.

В процессе сообщение разбивается последовательно по 512 бит:
for перебираем все такие части
 разбиваем этот кусок на 16 частей, слов по 32-бита (big-endian) w[i], 0 ≤ i ≤ 15

16 слов по 32-бита дополняются до 80 32-битовых слов:
for i from 16 to 79
 w[i] = (w[i-3] xor w[i-8] xor w[i-14] xor w[i-16]) циклический сдвиг влево 1

Инициализация хеш-значений этой части:
a = h0
b = h1
c = h2
d = h3
e = h4

Основной цикл:
for i from 0 to 79
 if 0 ≤ i ≤ 19 then
 f = (b and c) or ((not b) and d)
 k = 0x5A827999
 else if 20 ≤ i ≤ 39 then
 f = b xor c xor d
 k = 0x6ED9EBA1
 else if 40 ≤ i ≤ 59 then
 f = (b and c) or (b and d) or (c and d)
 k = 0x8F1BBCDC
 else if 60 ≤ i ≤ 79 then
 f = b xor c xor d
 k = 0xCA62C1D6

 temp = (a leftrotate 5) + f + e + k + w[i]
 e = d
 d = c
 c = b leftrotate 30
 b = a
 a = temp

Добавляем хеш-значение этой части к результату:
h0 = h0 + a
h1 = h1 + b
h2 = h2 + c
h3 = h3 + d
h4 = h4 + e

Итоговое хеш-значение(h0, h1, h2, h3, h4 должны быть преобразованы к big-endian):
digest = hash = h0 append h1 append h2 append h3 append h4

Вместо оригинальной формулировки FIPS PUB 180-1 приведены следующие эквивалентные выражения и могут быть использованы на компьютере f в главном цикле:

(0 ≤ i ≤ 19): f = d xor (b and (c xor d))
(0 ≤ i ≤ 19): f = (b and c) xor ((not b) and d)
(0 ≤ i ≤ 19): f = (b and c) + ((not b) and d)

(альтернатива 1)
(альтернатива 2)
(альтернатива 3)

(40 ≤ i ≤ 59): f = (b and c) or (d and (b or c))
(40 ≤ i ≤ 59): f = (b and c) or (d and (b xor c))
(40 ≤ i ≤ 59): f = (b and c) + (d and (b xor c))
(40 ≤ i ≤ 59): f = (b and c) xor (b and d) xor (c and d)

(альтернатива 1)
(альтернатива 2)
(альтернатива 3)
(альтернатива 4)

Примеры

Ниже приведены примеры хешей SHA-1. Для всех сообщений подразумевается использование кодировки UTF-8.

Хеш панграммы на русском:

SHA-1("В чашах юга жил бы цитрус? Да, но фальшивый экземпляр!")
= 9e32295f 8225803b b6d5fdfc c0674616 a4413c1b

Хеш панграммы на английском:

SHA-1("The quick brown fox jumps over the lazy dog")
= 2fd4e1c6 7a2d28fc ed849ee1 bb76e739 1b93eb12

SHA-1("sha")
= d8f45903 20e1343a 915b6394 170650a8 f35d6926

Небольшое изменение исходного текста (одна буква в верхнем регистре) приводит к сильному изменению самого хеша. Это происходит вследствие лавинного эффекта.

SHA-1("Sha")
= ba79baeb 9f10896a 46ae7471 5271b7f5 86e74640

Даже для пустой строки вычисляется нетривиальное хеш-значение.

SHA-1("")
= da39a3ee 5e6b4b0d 3255bfef 95601890 afd80709

Криптоанализ

Криптоанализ хеш-функций направлен на исследование уязвимости для различного вида атак. Основные из них:

- нахождение коллизий — ситуация, когда двум различным исходным сообщениям соответствует одно и то же хеш-значение.
- нахождение прообраза — исходного сообщения — по его хешу.

При решении методом «грубой силы»:

- первая задача требует в среднем $2^{160/2} = 2^{80}$ операций, если использовать атаку Дней рождения.
- вторая требует 2^{160} операций.

От устойчивости хеш-функции к нахождению коллизий зависит безопасность электронной цифровой подписи с использованием данного хеш-алгоритма. От устойчивости к нахождению прообраза зависит безопасность хранения хешей паролей для целей аутентификации.

В январе 2005 года Винсент Рэймен и Elisabeth Oswald опубликовали сообщение об атаке на усечённую версию SHA-1 (53 раунда вместо 80), которая позволяет находить коллизии меньше, чем за 2^{80} операций.

В феврале 2005 года Сяююнь Ван, Ицюнь Лиза Инь и Хунбо Юй (Xiaoyun Wang, Yiqun Lisa Yin, Hongbo Yu) представили атаку на полноценный SHA-1, которая требует менее 2^{69} операций.

О методе авторы пишут:

Мы представляем набор стратегий и соответствующих методик, которые могут быть использованы для устранения некоторых важных препятствий в поиске коллизий в SHA-1. Сначала мы ищем близкие к коллизии дифференциальные пути, которые имеют небольшой «вес Хамминга» в «векторе помех», где каждый бит представляет 6-шаговую локальную коллизию. Потом мы соответствующим образом корректируем дифференциальный путь из первого этапа до другого приемлемого дифференциального пути, чтобы избежать неприемлемых последовательных и усеченных коллизий. В конце концов мы преобразуем два одноблоковых близких к коллизии дифференциальных пути в один двухблоковый коллизионный путь с удвоенной вычислительной сложностью.^[1]

Оригинальный текст (англ.) [\[показать\]](#)

Также они заявляют:

В частности, наш анализ основан на оригинальной дифференциальной атаке на SHA-0, «near-collision» атаке на SHA-0, мультиблоковой методике, а также методике модификации исходного сообщения, использованных при атаках поиска коллизий на HAVAL-128, MD4, RIPEMD и MD5.

Оригинальный текст (англ.) [\[показать\]](#)

Статья с описанием алгоритма была опубликована в августе 2005 года на конференции CRYPTO.

В этой же статье авторы опубликовали атаку на усечённый SHA-1 (58 раундов), которая позволяет находить коллизии за 2^{33} операций.

В августе 2005 года на CRYPTO 2005 эти же специалисты представили улучшенную версию атаки на полноценный SHA-1, с вычислительной сложностью в 2^{63} операций. В декабре 2007 года детали этого улучшения были проверены Мартином Кохраном.

Кристоф де Каньер и Кристиан Рехберг позже представили усовершенствованную версию атаки на SHA-1, за что были удостоены награды за лучшую статью на конференции ASIACRYPT 2006. Ими была представлена двухблоковая коллизия на 64-раундовый алгоритм с вычислительной сложностью около 2^{35} операций.^[2]

Существует масштабный исследовательский проект, стартовавший в технологическом университете австрийского города Грац, который : «... использует компьютеры, соединённые через Интернет, для проведения исследований в области криптоанализа. Вы можете поучаствовать в проекте, загрузив и запустив бесплатную программу на своем компьютере.»^[3]

Бурт Калински, глава исследовательского отдела в «лаборатории RSA», предсказывает, что первая атака по нахождению прообраза будет успешно осуществлена в ближайшие 5—10 лет.

Ввиду того, что теоретические атаки на SHA-1 оказались успешными, NIST планирует полностью отказаться от использования SHA-1 в цифровых подписях.^[4]

Из-за блочной и итеративной структуры алгоритмов, а также отсутствия специальной обработки в конце хеширования, все хеш-функции семейства SHA уязвимы для атак удлинением сообщения и коллизиям при частичном хешировании сообщения.^[5] Эти атаки позволяют подделывать сообщения, подписанные только хешем — *SHA(message||key)* или *SHA(key||message)* — путём удлинения сообщения и пересчёту хеша без знания значения ключа. Простейшим исправлением, позволяющим защититься от этих атак, является двойное хеширование — *SHA_d(message) = SHA(SHA(0^b||message))* (0^b — блок нулей той же длины, что и блок хеш-функции).

2 ноября 2007 года NIST анонсировало конкурс по разработке нового алгоритма SHA-3, который продлился до 2012 года.^[6]

SHAppening

8 октября 2015 Marc Stevens, Pierre Karpman, и Thomas Peyrin опубликовали атаку на функцию сжатия алгоритма SHA-1, которая требует всего 2^{57} вычислений.

Реальный взлом: SHAttered (нахождение коллизий)

23 февраля 2017 года специалисты из Google и CWI объявили о практическом взломе алгоритма, опубликовав 2 PDF-файла с одинаковой контрольной суммой SHA-1. Это потребовало перебора 9×10^{18} вариантов, что заняло бы 110 лет на 1 GPU^[7].

Сравнение SHA-1 с другими алгоритмами

Сравнение с MD5

И MD5, и SHA-1 являются, по сути, улучшенными продолжениями MD4.

Сходства:

- Четыре этапа.
- Каждое действие прибавляется к ранее полученному результату.
- Размер блока обработки, равный 512 бит.
- Оба алгоритма выполняют сложение по модулю 2^{32} , они рассчитаны на 32-битную архитектуру.

Различия:

- 1. В SHA-1 на четвёртом этапе используется та же функция *f*, что и на втором этапе.
- 2. В MD5 в каждом действии используется уникальная прибавляемая константа. В SHA-1 константы используются повторно для каждой из четырёх групп.
- 3. В SHA-1 добавлена пятая переменная.
- 4. SHA-1 использует циклический код исправления ошибок.
- 5. В MD5 четыре сдвига, используемые на каждом этапе, отличаются от значений, используемых на предыдущих этапах. В SHA на каждом этапе используется постоянное значение сдвига.
- 6. В MD5 — четыре различных элементарных логических функции, в SHA-1 — три.
- 7. В MD5 длина дайджеста составляет 128 бит, в SHA-1 — 160 бит.
- 8. SHA-1 содержит больше раундов (80 вместо 64) и выполняется на 160-битном буфере по сравнению со 128-битным буфером MD5. Таким образом, SHA-1 должен выполняться приблизительно на 25 % медленнее, чем MD5 на той же аппаратуре.

Брюс Шнайер делает следующий вывод : «SHA-1 — это MD4 с добавлением расширяющего преобразования, дополнительного этапа и улучшенным лавинным эффектом. MD5 — это MD4 с улучшенным битовым хешированием, дополнительным этапом и улучшенным лавинным эффектом.»

Сравнение с ГОСТ Р 34.11-94

Ряд отличительных особенностей ГОСТ Р 34.11-94:

- 1. При обработке блоков используются преобразования по алгоритму ГОСТ 28147—89;
- 2. Обрабатывается блок длиной 256 бит, и выходное значение тоже имеет длину 256 бит.
- 3. Применены меры борьбы против поиска коллизий, основанном на неполноте последнего блока.
- 4. Обработка блоков происходит по алгоритму шифрования ГОСТ 28147—89, который содержит преобразования на S-блоках, что существенно осложняет применение метода дифференциального криптоанализа к поиску коллизий алгоритма ГОСТ Р 34.11-94. Это существенный плюс по сравнению с SHA-1.
- 5. Теоретическая криптостойкость ГОСТ Р 34.11-94 равна 2^{128} , что во много раз превосходит 2^{80} для SHA-1.

Сравнение с другими SHA

В таблице «промежуточный размер хеша» означает «размер внутренней хеш-суммы» после каждой итерации.

Вариации алгоритма		Размер выходного хеша (бит)	Промежуточный размер хеша (бит)	Размер блока (бит)	Максимальная длина входного сообщения (бит)	Размер слова (бит)	Количество раундов	Используемые операции	Найденные коллизии
SHA-0		160	160	512	$2^{64} - 1$	32	80	+,and, or, xor, rotl	Есть
SHA-1		160	160	512	$2^{64} - 1$	32	80	+,and, or, xor, rotl	2^{52} операций
SHA-2	SHA-256/224	256/224	256	512	$2^{64} - 1$	32	64	+,and, or, xor, shr, rotr	Нет
	SHA-512/384	512/384	512	1024	$2^{128} - 1$	64	80	+,and, or, xor, shr, rotr	Нет

Использование

Хеш-функции используются в системах контроля версий, системах электронной подписи, а также для построения кодов аутентификации.

SHA-1 является наиболее распространенным из всего семейства SHA и применяется в различных широко распространенных криптографических приложениях и алгоритмах.

SHA-1 используется в следующих приложениях:

- S/MIME — дайджесты сообщений.
- SSL — дайджесты сообщений.
- IPSec — для алгоритма проверки целостности в соединении «точка-точка».
- SSH — для проверки целостности переданных данных.

- **PGP** — для создания электронной цифровой подписи.
- **Git** — для идентификации каждого объекта по SHA-1-хешу от хранимой в объекте информации.
- **Mercurial** — для идентификации ревизий
- **BitTorrent** — для проверки целостности загружаемых данных.

SHA-1 является основой блочного шифра **SHACAL**.

Отказ от использования

Компания **Google** давно выразила своё недоверие SHA-1, особенно для использования этой функции для подписи сертификатов **TLS**. Ещё в 2014 году, вскоре после публикации работы Марка Стивенса, группа разработчиков **Chrome** объявила о постепенном отказе от использования SHA-1.

С 25 апреля 2016 года **Яндекс**. Почта перестала поддерживать старые почтовые программы, использующие SHA-1^[8].

Примечания

- Finding Collisions in the Full SHA-1 (<https://www.webcitation.org/61A39qlpS?url=http://www.infosec.sdu.edu.cn/uploadfile/papers/Finding%20Collisions%20in%20the%20Full%20SHA-1.pdf>) (англ.). — Статья китайских исследователей о взломе SHA-1. Архивировано из оригинала (<http://www.infosec.sdu.edu.cn/uploadfile/papers/Finding%20Collisions%20in%20the%20Full%20SHA-1.pdf>) 23 августа 2011 года.
- Finding SHA-1 Characteristics: General Results and Applications (https://dx.doi.org/10.1007/11935230_1) (англ.). Дата обращения: 4 октября 2017. Архивировано (https://web.archive.org/web/20080726102016/http://dx.doi.org/10.1007/11935230_1) 26 июля 2008 года.
- SHA-1 Collision Search Graz (<http://boinc.iaik.tugraz.at>) (англ.). — Исследовательский проект технологического университета Граца. Архивировано (<https://web.archive.org/web/20081107231039/http://boinc.iaik.tugraz.at/>) 7 ноября 2008 года.
- NIST Comments on Cryptanalytic Attacks on SHA-1 (<https://www.webcitation.org/6BNBystHR?url=http://csrc.nist.gov/groups/ST/hash/statement.html>) (англ.). — Официальный комментарий NIST по поводу атак на SHA-1. Архивировано из оригинала (<http://csrc.nist.gov/groups/ST/hash/statement.html>) 13 октября 2012 года.
- Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno, Cryptography Engineering (<https://www.webcitation.org/6BNBzP5Ci?url=http://www.schneier.com/book-ce.html>) (англ.). Архивировано из оригинала (<http://www.schneier.com/book-ce.html>) 13 октября 2012 года., John Wiley & Sons, 2010. ISBN 978-0-470-47424-2
- NIST Hash Competition (https://www.webcitation.org/6BNBzvQEO?url=http://csrc.nist.gov/groups/SMA/ispab/document/minutes/2008-06/HashCompetition-June2008_BBurr-JKelsey.pdf) (англ.). — Конкурс на разработку SHA-3. Архивировано из оригинала (http://csrc.nist.gov/groups/SMA/ispab/documents/minutes/2008-06/HashCompetition-June2008_BBurr-JKelsey.pdf) 13 октября 2012 года.
- Первый способ генерации коллизий для SHA-1 (<https://habrahabr.ru/post/322478/>). Дата обращения: 9 марта 2017. Архивировано (<https://web.archive.org/web/20170310042133/https://habrahabr.ru/post/322478/>) 10 марта 2017 года.
- Обновление почтовых программ — Почта — Яндекс.Помощь (<https://yandex.ru/support/mail/mail-clients/sha-256.xml>). yandex.ru. Дата обращения: 7 апреля 2016. Архивировано (<https://web.archive.org/web/20160420070950/http://yandex.ru/support/mail/mail-clients/sha-256.xml>) 20 апреля 2016 года.

Литература

- *Шнайер Б.* Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си = Applied Cryptography. Protocols, Algorithms and Source Code in C. — М.: Триумф, 2002. — 816 с. — 3000 экз. — ISBN 5-89392-055-4.
- *Нильс Фергюсон, Брюс Шнайер.* Практическая криптография = Practical Cryptography: Designing and Implementing Secure Cryptographic Systems. — М. : Диалектика, 2004. — 432 с. — 3000 экз. — ISBN 5-8459-0733-0, ISBN 0-4712-2357-3.

Ссылки

- **RFC 3174** (англ.)
- Обзор SHA-1 от Консорциума Всемирной паутины (http://www.w3.org/PICS/DSig/SHA1_1_0.html) Архивная копия (https://web.archive.org/web/20050304074240/http://www.w3.org/PICS/DSig/SHA1_1_0.html) от 4 марта 2005 на **Wayback Machine** (англ.)
- Взлом SHA-1 (http://www.schneier.com/blog/archives/2005/02/sha1_broken.html) Архивная копия (https://web.archive.org/web/20170505152637/http://www.schneier.com/blog/archives/2005/02/sha1_broken.html) от 5 мая 2017 на **Wayback Machine** (англ.)
- Доклад о взломе SHA1 (<https://www.webcitation.org/61A39qlpS?url=http://www.infosec.sdu.edu.cn/uploadfile/papers/Finding%20Collisions%20in%20the%20Full%20SHA-1.pdf>) (англ.)

- Брюс Шнайер о взломе SHA (http://www.schneier.com/blog/archives/2005/02/cryptanalysis_o.html) Архивировано (http://www.webcitation.org/6CaZxHamO?url=http://www.schneier.com/blog/archives/2005/02/cryptanalysis_o.html) 1 декабря 2012 года. (англ.)
- Последствия удачных атак на SHA-1 (<http://www.heise-online.co.uk/security/Hash-cracked--/features/75686/0>) Архивная копия (<https://web.archive.org/web/20081226212806/http://www.heise-online.co.uk/security/Hash-cracked--/features/75686/0>) от 26 декабря 2008 на [Wayback Machine](#) (англ.)

Источник — <https://ru.wikipedia.org/w/index.php?title=SHA-1&oldid=123378860>

Эта страница в последний раз была отредактирована 18 июня 2022 в 17:14.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)